



University
Mohammed VI
Polytechnic



Lab 2: Relational Schema

Data Management Course

UM6P College of Computing

Professor: Karima Echihabi **Program:** Computer Engineering

Session: Fall 2026

Team Information

Member 1	Name: BENYASSINE Rayane
Member 2	Name: AIT LAHCEN Adam
Member 3	Name: BITAR Driss
Repository Link	https://github.com/Adam-Ait-Lahcen/Base-de-donn-es

1 Introduction

This lab focuses on translating the final RLMS conceptual design developed in Lab 1 into a relational schema and implementing it using SQL. The main objective is to apply the ER-to-relational mapping rules studied in the course to the different constructs of the ER diagram, while defining the corresponding primary keys, foreign keys, composite keys, and integrity constraints.

2 Requirements

- Map all entity sets and relationship sets of the RLMS ER diagram into relations..
- Produce an SQL file containing the statements required to convert the ER diagram into relational tables.
- Provide a short report explaining the roles of the team members, the main design choices, the assumptions, and the challenges faced.

3 Team Roles

The three members of the team, Adam AIT LAHCEN, Driss BITAR, and Rayane BENYASSINE, worked together on the relational schema and the SQL implementation. Although some parts of the solution were mainly handled by one member, the final work was collective. The different modeling choices, keys, foreign keys, and SQL statements were discussed and reviewed by the three members before being included in the final solution. Driss was mainly responsible for the logical design of the **Person hierarchy and its related relationships** as well as the **Consumable, Supplier, Stocks, supplies, and consumes** part. Adam was mainly responsible for the logical design of **Equipment and Certifications**, including, he also mainly worked on the weak entities **Maintenance** and **CalibrationRecord**. Rayane was mainly responsible for the logical design of **Laboratories, Research Projects, and Budgets** and as well as their associated relationships and also responsible for writing the report. However, the report was reviewed and revised and modified by the other two members before submission. These responsibilities were only used to organize the work. No part of the final solution was considered individual work, since the complete schema and SQL code were collectively reviewed and validated by the group.

4 Justification of the main choices

The relational schema was obtained by directly applying the ER-to-relational mapping rules studied in the course. For the **Person hierarchy**, we used the table-per-type approach. Each subtype is represented by a separate relation whose primary key is also a foreign key referencing its direct parent. For example, Student(student_id) references Person(pid), while Academic(academic_id) references Employee(employee_id). This allows the common attributes to remain in Person while preserving the different levels of the ISA hierarchy. The Advises relationship between Student and Academic

is a **many-to-many relationship**. Therefore, it was represented using a separate table containing the two foreign keys, with (student_id, academic_id) as its composite primary key. The same rule was applied to Attached, which connects Laboratory and Person, and to Participates, which connects Person and Research_Project. In Participates, pRole is kept in the relationship table because it describes the role of a particular person in a particular project. The same mapping principle was used for FundsLab and FundsPrj, which connect budgets to laboratories and research projects. For **one-to-many relationships**, the foreign key was placed on the many side. For example, faculty_supervisor is included in Laboratory for Supervises, while managerID is included in Budget for Manages. Similarly, equipmentModel and located_in are included in EquipmentUnit for the InstanceOf and LocatedIn relationships. In Reservation, made_by, project_code, and approver represent the corresponding relationships with Person and Research Project. The ReportsTo relationship is **recursive**, since an employee can report to another employee. It was therefore represented by the self-referencing foreign key supervisor inside Employee. For Equipment and Certifications, requires is a **many-to-many relationship** between EquipmentModel and Certification, so it was represented by a separate relation with a composite primary key. Similarly, holds connects Person and Certification. Its attributes issueDate, expirationDate, and grade are stored in the relationship table because they describe a specific certification held by a specific person. The reserves relationship between Reservation and EquipmentUnit was also represented by a separate table with a composite primary key, following the mapping rule for a many-to-many relationship. Maintenance and CalibrationRecord were represented as **weak entities** because they depend on an EquipmentUnit for their identification. Their primary keys therefore combine the equipment unit identifier with their respective partial discriminator: (equipmentUnit, startTs) for Maintenance and (equipmentUnit, calibDate) for CalibrationRecord. For Consumables and Stock, Stocks represents the association between a Consumable and a Laboratory, so (consId, lab_id) forms its composite primary key. The monitoring information is stored directly in this relation through monitor_by and monitoring_since. Finally, supplies represents the **ternary relationship** between Consumable, Laboratory, and Supplier. The three participating identifiers therefore form its composite primary key, while unitPrice is kept as an attribute of the relationship. The consumes relation connects a Reservation to a stock identified by (consId, lab_id), which is why it uses this pair as a composite foreign key.

5 Assumptions

We did not introduce any particular assumptions beyond those already established in the requirements and the conceptual design. The main additional consideration concerns the SQL data types: identifiers were generally represented using the INT data type, while identifiers that naturally represent textual codes, such as budgetLine, were represented using VARCHAR.

6 Conclusion

Overall, the implementation was a direct application of the material covered in the course. The final respond was reviewed collectively before submission.